```
Terminal
satya:~$ cc blog2.c
blog2.c: In function 'fun':
blog2.c:6:2: warning: function returns address of local variable [-Wreturn-local
-addr]
  return (&auto_var);
  ^
```

Figure 1: Compiler generating warning

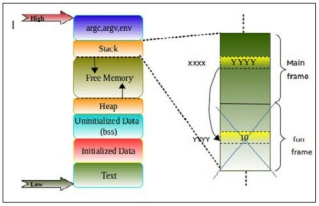

Figure 2: Program memory and stack frames

```
 9          return 0;
10 }
11
12 int *fun(void)
13 {
14          int auto_var = 10;
15          return (&auto_var);
16 }
```

When we run the program shown in Code 1 in the GCC compiler, we get the warning shown in Figure 1.

What is wrong when the address of the auto/local variables is returned? Let us examine this in detail:

1. We know that whenever there is a function call, a new stack frame will be created automatically, where *auto variable auto_var* is local in our example. Its scope and lifetime is within the function call.
2. When the control returns from the function call, all the memory allocated for that function will be freed automatically.
3. In our example program, we are returning the address of the auto variable and collecting this in the *int_ptr* pointer in the main function. So, *int_ptr* is still pointing to the memory, which is freed as mentioned in Point 2.
4. One can observe allocation and deallocation of the stack frame (let us called this *fun frame*) for the *fun()* in Figure 2 for better understanding.
5. Now, *int_ptr* becomes a dangling pointer. Dereferencing this pointer results in *unexpected output.*
6. So, always take extra care while playing with pointers and local variables.

Any attempt to dereference the pointer that is already dangling may still print the correct value after the control returning from a function call, but any functions called thereafter will overwrite the stack storage allocated for the *auto_var* variable with other values, and the pointer will no longer work correctly.

How to prevent a pointer from becoming a dangling pointer in this case: If a pointer to *auto_var* is to be returned, *auto_var* must have scope beyond the function so that it may be declared as static in order to avoid the pointer from dangling. This is because the memory allocated to the static variables is from the data segment, where the lifetime will be throughout the program.

Case 2: When the variable goes out of scope

Consider the sample code (Code 2) given below for analysis:

```
 1 #include <stdio.h>
 2
 3 int main()
 4 {
 5          int *iptr;
 6          //Block started
 7          {
 8                  int var = 10;
 9                  iptr = &var;
10          } //After this block iptr is dangling
11          //Some code goes here
12          return 0;
13 }
```

Running the program shown in Code 2 in the GCC compiler with the *-Wall* option results in the warning shown in Figure 3.

Since the variable *var* is invisible for the outer block shown in Code 2, *iptr* is still pointing to the same object even when the control comes out of the inner block. Hence, the pointer *iptr* becomes a dangling pointer after Line 10 in the example shown in Code 2.

Case 3: When, in dynamic memory allocation, the block of memory that is already freed is used

Consider the sample code given below:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main()
6 {
7 int *block_ptr = (int *) malloc(sizeof(int));
8
9 //Do something with allocated memory
10
11 free(block_ptr);
```